```
Output:
I divided by zero. :( caught in generic Error handler
```

DIY Exceptions

The technique of specifying multiple `catch` blocks with differing type declarations allows you to fine-tune your exception handling based purely on the class of exception being thrown. You can easily create specialized exceptions for any problem case arising in your code and use them to switch between `catch` blocks accordingly. To define your own class, just use the `class` and `extends` keywords as previously demonstrated:

```
class MyException extends Exception
{
    // extra functionality here if required
}
```

All class declarations have a block (delimited by `{ }` braces) into which you can place additional code, but that's rarely required when making subclasses of `Exception` (which generally has enough functionality already). It's very common to see a slew of exception classes in third-party libraries that do nothing but extend another exception and define zero extra functionality: it's the class itself that's the important thing. One more thing to mention: classes can be namespaced just as you've seen in Chapter 3, "Functions," though I've not shown it here, and class namespacing is fully explained in Chapter 6, "Object Fundamentals." Namespacing gives you yet more options for specializing your exceptions (and avoiding name collisions, of course). If you are in a non-root namespace, don't forget to prefix `Exception` and `Error` with a backslash (the reasons why are explained in Chapter 3, "Functions") – for example, `catch (\Exception $e) { ... }` or `class SomeNamespacedClass extends \Exception`.

Finally…

There is an additional component to the `try/catch` construct, a way to define code that will always be executed in the event of an exception (irrespective of its type). The `finally { }` block must go at the end of any `catch` blocks, and it can even be used instead of `catch` blocks (a `try` must have at least one `catch` or `finally`). A `finally` block always executes, no matter if the exception is caught or not: